

nmpc_simulation.py 詳細解説レポート（最新版）

対象ファイル: `nmpc_simulation.py` (1566 行)
更新日: 2026-05-25 (U_ref LPF $\tau=0.1$ s 反映)
目的: Hexa-X 配置ヘキサコプタの耐故障姿勢+高度制御を NMPC (ADMM + SR2) で検証する Python シミュレータ
参照実装: `sample/nmpc.cpp`, `nmpc_model.cpp`, `nmpc_params.hpp` (Pico 2 移植前オフライン検証用)

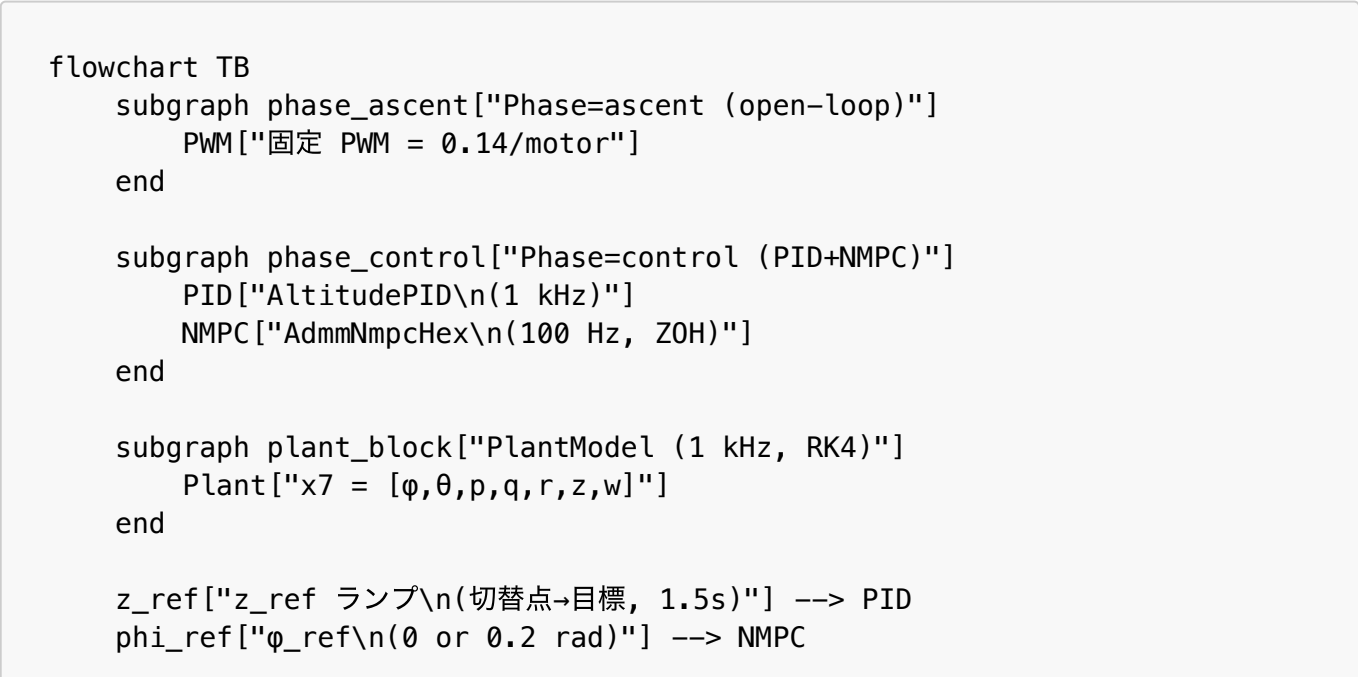
1. 概要

6 モータ・5 状態 (ϕ, θ, p, q, r) の NMPC を凝縮形で解き、プラントを **1 kHz** で積分しながら NMPC は **100 Hz** (ZOH) で更新する **マルチレート** 構成。

高度制御・離陸シーケンスも組み込まれており、以下の機能が統合されている。

機能	概要
Open-loop 離陸	地上から目標-30 cm まで固定 PWM で上昇 (PID 飽和回避)
高度 PID	重力補償付き PID + 推力下限クリップ → <code>U_ref</code> 経由で NMPC の動作点を動的調整
マルチレート	プラント・PID: 1 kHz / NMPC: 100 Hz (ZOH)
Λ スケーリング	$B_{eff} = B_{nom} \cdot \text{diag}(\eta)$ で FDI 既知時のモデル適応 (<code>u_max</code> は二重縮減しない)
外乱注入	world 力・body トルクのスケジュール
絶対時刻イベント	PID 切替時刻基準で故障注入・roll ステップを決定論的に同期
U_ref LPF	PID 出力の per-motor duty を 1 次 LPF ($\tau=0.1$ s) して NMPC 動作点の急変を抑制

2. アーキテクチャ



```
Plant -->|x7[5,6]: z,w| PID
Plant -->|x7[:5]: 姿勢| NMPC
PID -->|"T_cmd → u_hover_raw"| LPF["U_ref LPF\nτ=0.1s"]
LPF -->|"U_ref"| NMPC
PWM -->|ascent 中| Plant
NMPC -->|u_cmd (ZOH)| Mult["u_applied = η ⊙ u_cmd"]
FM["FaultManager\n(apply_faults)"] --> Mult
DM["DisturbanceManager"] -->|F_world, τ_body| Plant
Mult --> Plant
```

フェーズ遷移 (run_scenario 内)

```
t=0  z=0 (地上)
|
| [phase = ascent] 全機 PWM=0.14 固定、NMPC は動作するが出力無視
|
▼ z ≤ z_switch = z_target + 0.30 m (= -1.70 m)
| → alt_pid.reset(), u_hover_prev ← ascent PWM、t_switch 記録
|
| [phase = control] PID + NMPC 通常動作 (U_ref は LPF 後)
| z_ref: z_at_switch → z_target に 1.5 s 線形ランプ
|
▼ t = t_switch + 2.0 s
| → plan_faults 適用 (S1: なし, S2a: M5 η=0.5, ...)
| → FDI 既知時: B_eff 更新, box 再設定, SR2 リセット
|
▼ t = t_fault + 5.0 s
| → phi_ref = 0.2 rad ステップ (roll 応答観測)
```

3. 座標系と記号

3.1 NED (North-East-Down)

変数	符号	意味
z	下向き正 [m]	地上 z=0、高度 2 m → z = -2.0
w	下向き正 [m/s]	落下で $w > 0$
T_cmd	[N]	機体推力総和コマンド (上向き)

プロットの「高度」は **-z** (上向き正) で表示。

3.2 Hexa-X モータ配置

モータ	方位	回転方向	x_motor [m]	y_motor [m]
M1	+90° (右横)	CW (s=-1)	0	+0.05

モータ	方位	回転方向	x_motor [m]	y_motor [m]
M2	-90° (左横)	CCW (s=+1)	0	-0.05
M3	-30° (前左)	CW (s=-1)	+0.043	-0.025
M4	+150° (後右)	CCW (s=+1)	-0.043	+0.025
M5	+30° (前右)	CCW (s=+1)	+0.043	+0.025
M6	-150° (後左)	CW (s=-1)	-0.043	-0.025

4. モジュール構成一覧

#	クラス / 関数	行	役割
—	make_run_img_dir	54	img/<timestamp>/ 作成
—	_cholesky_lower	66	np.linalg.cholesky ラッパ
1	Params	74	全パラメータ dataclass
2	HexaModel	148	姿勢 CT モデル・Jacobian・離散化・ Λ スケーリング
2.5	PlantModel	290	7 状態プラント RK4
2.6	AltitudePID	359	PID + 重力補償 + アンチwindアップ + T_cmd 下限
—	t_cmd_to_u_hover	414	T_cmd → per-motor duty (生存機均等配分)
2.7	DisturbanceManager	434	カ/トルク外乱スケジュール
3	AdmmNmpcHex	470	凝縮 NMPC コア (Sx/Su/H, TR1, SR2, ADMM)
4	FaultManager	707	$\eta(t)$ + FDI イベント + apply_faults
5	Logger	757	1 kHz ログ + 5×4 プロット
6	sanity_checks	~960	起動時 13 項目自己診断
7	run_scenario	1167	マルチレート + フェーズ制御 + U_ref LPF
8	main	1417	S1~S8 一括実行

5. Params (パラメータ)

5.1 物理パラメータ

記号	変数	値	意味
m	mass	0.200 kg	機体質量
g	gravity	9.81 m/s ²	重力
Ix/Iy/Iz	各慣性	6.10e-3 / 6.53e-3 / 1.16e-2 kg·m ²	主慣性モーメント

記号	変数	値	意味
L	arm_length	0.050 m	モーター-重心距離
T_max	T_max	2.725 N	1 モータ最大推力
C_QT	C_QT	0.03614 m	反トルク/推力比
Cp/Cq/Cr	減衰	0.01/0.025/0.005	smooth

5.2 NMPC・マルチレート

変数	値	説明
N	20	予測ホライズン段数
dt	0.01 s	NMPC 予測モデル の離散化周期（Sx/Su/H に使用、変更不可）
dt_sim	0.001 s	プラント RK4・PID 周期（1 kHz）
nmpe_decimation	10	NMPC 呼出間隔（実効 10 ms = 100 Hz）
NX, NU	5, 6	状態・入力次元
U_min/U_max	0.0 / 1.0	モータ duty 下限・上限

5.3 高度 PID（再チューニング済）

変数	値	旧値	変更理由
z_ref_default	-2.0 m	-1.0 m	2 m ホバー目標
Kp_z	3.0	4.0	過補償低減
Ki_z	0.3	0.5	積分暴走防止
Kd_z	2.5	3.0/4.0	切替時 derivative kick 抑制
i_z_limit	0.2 N	0.5 N	mg の ~10 % に縮小
T_cmd_min_ratio	0.3	(なし)	推力下限 = 0.3・mg（自由落下防止）
trim_clip_cos	0.5	0.5	変更なし

5.4 Open-loop 離陸フェーズ

変数	値	説明
pwm_ascent	0.14	上昇中の per-motor duty（ホバー 0.12 の 17 % 増）
z_switch_offset	0.30 m	目標より 30 cm 手前（NED 正方向）で PID 切替
z_ramp_duration	1.5 s	PID 切替後 z_ref を z_at_switch → z_target にランプ接続する時間
u_ref_lpf_tau	0.10 s	U_ref 用 1 次 LPF の時定数（0 で無効=通過）

5.5 U_ref 低域通過フィルタ（LPF）

高度 PID の出力 T_{cmd} を $t_{cmd_to_u_hover}$ で 6 モータ duty u_hover_raw に変換したあと、**NMPC の U_{ref} に載せる前に 1 次 LPF をかける。**

離散更新 ($dt_{sim} = 1\text{ ms}$) :

```
α = dt_sim / (τ + dt_sim)      # τ=0.10 → α ≈ 0.0099
u_hover = α · u_hover_raw + (1-α) · u_hover_prev
u_hover_prev ← u_hover
U_ref = tile(u_hover, N)
```

設計意図	内容
目的	T_{fb} の急変が NMPC 動作点を毎ステップ揺らし、ADMM 1 反復では追従しきれない姿勢発散を抑制
切替時	ascent→control で $u_hover_prev \leftarrow pwm_ascent$ (LPF ステップ防止)
ascent 中	毎周期 $u_hover_prev \leftarrow ascent_pwm_vec$ (PID 起動時の不連続を回避)
故障時	PID 積分・LPF はリセットしない (定常ホバー中の i_z を維持。過去の故障時リセットは S3a/S6/S7 で悪化したため撤廃)

5.6 コスト・RTI

変数	値	旧値	説明
Q_{ϕ}/Q_{θ}	100	100	姿勢角コスト
$Q_p/Q_q/Q_r$	1	1	角速度コスト
R_u	10.0	17.0	入力ペナルティ (NMPC を機敏に)
$terminal_scale$	5	5	終端段の Q 倍率
$reset_period$	100	100	フル再構築間隔 [NMPC 周期]
$admm_max_iter$	1	1	ADMM 反復回数 (実時間向け 1 回)
$admm_rho$	1.0	1.0	ADMM ペナルティ

プロパティ: $u_hover_nom = m \cdot g / (6 \cdot T_{max}) \approx 0.120$

6. HexaModel (姿勢ダイナミクス)

6.1 連続時間モデル

状態 $x = [\phi, \theta, p, q, r]$ 。

運動学 (θ にクランプ付き) :

$$\begin{aligned}\dot{\phi} &= p + q \cdot \sin(\phi) \cdot \tan(\theta) + r \cdot \cos(\phi) \cdot \tan(\theta) \\ \dot{\theta} &= q \cdot \cos(\phi) - r \cdot \sin(\phi)\end{aligned}$$

剛体回転（入力トルク + ジャイロ + smooth $|v|$ 減衰）：

$$\begin{aligned}\dot{p} &= (B_{\text{eff}} \cdot u)[x] + (-(I_z - I_y) \cdot q \cdot r - C_p \cdot p \cdot |p| \varepsilon) / I_x \\ \dot{q} &= (B_{\text{eff}} \cdot u)[y] + (-(I_x - I_z) \cdot r \cdot p - C_q \cdot q \cdot |q| \varepsilon) / I_y \\ \dot{r} &= (B_{\text{eff}} \cdot u)[z] + (-(I_y - I_x) \cdot p \cdot q - C_r \cdot r \cdot |r| \varepsilon) / I_z\end{aligned}$$

$|v| \varepsilon = \sqrt{(v^2 + \varepsilon^2)}$ (`damping_eps = 1e-3`) で C^2 化し Jacobian も解析的に計算。

6.2 B_c 行列 (5×6)

$$\begin{aligned}B_c[2,k] &= -y_k \cdot T_{\text{max}} / I_x && (\text{ロールトルク}) \\ B_c[3,k] &= +x_k \cdot T_{\text{max}} / I_y && (\text{ピッチトルク}) \\ B_c[4,k] &= s_k \cdot C_{QT} \cdot T_{\text{max}} / I_z && (\text{ヨートルク})\end{aligned}$$

上 2 行 (ϕ, θ への直接入力項) はゼロ。

6.3 $\wedge$ スケーリング (耐故障の核心)

$$B_{\text{eff}} = B_{\text{nom}} \cdot \text{diag}(\eta) \quad \# \text{ 列 } k \text{ に } \eta_k \text{ を乗算}$$

- `Bc_nom`: 健全時の 5×6 行列 (起動時に一度だけ構築)
- `set_eta(eta)`: FDI 既知時に NMPC 予測モデルを更新
- プラントの `PlantModel.attitude` は $\eta=1$ 固定 (故障は $u_{\text{applied}} = \eta \odot u_{\text{cmd}}$ 側で表現)

重要な設計修正 (今バージョン) : $u_{\text{max}}[k] = \eta_k$ という縮減は廃止。

- 旧来 B_{eff} 縮減 + u_{max} 縮減の二重スケーリングにより FDI 既知が未知より悪化していた
- 新: $\eta=0$ (完全故障) のみ $u_{\text{max}}[k] = 0$ 、それ以外は $u_{\text{max}}[k] = 1.0$ を維持

7. PlantModel (7 状態プラント)

7.1 状態

$$x_7 = [\phi, \theta, p, q, r, z, w]$$

- 姿勢 5 次元: `HexaModel.f` + body トルク外乱
- 並進 2 次元: 鉛直のみ (水平位置・ ψ は未モデル化)

7.2 鉛直ダイナミクス (NED)

総推力 $T_{total} = T_{max} \cdot \sum u_i$ (上向き) :

$$\begin{aligned}\dot{z} &= w \\ \dot{w} &= g - T_{total} \cdot \cos(\varphi) \cdot \cos(\theta) / m + F_{world,z} / m\end{aligned}$$

$\cos(\varphi)\cos(\theta)$ は $trim_clip_cos = 0.5$ で下限クリップ (大傾斜時の発散防止)。

7.3 外乱の入り方

種類	作用点
$\tau_{body}[0,1,2]$	$\dot{p}, q, \dot{r}$ に τ/l を加算 (姿勢乱れ)
$F_{world}[2]$	$\dot{w}$ に F/m を加算 (高度乱れ)
$F_{world}[0,1]$	水平外力 (並進 x, y なし → 現実装では未使用)

積分は RK4、1 ms ステップ内で入力・外乱は ZOH。

8. AltitudePID と U_ref 連携

8.1 制御則 (NED 整合版)

```
e_z = z - z_ref # 正 = 機体が目標より低い (落下方向にずれ)
i_z ← clip(i_z + e_z·dt, ±i_z_limit) # サチュ時は積分停止

T_trim = m·g / max(cos(φ)·cos(θ), 0.5) # 姿勢補償
T_fb = Kp·e_z + Ki·i_z + Kd·(w - w_ref)
T_cmd = max(T_trim + T_fb, T_cmd_min) # 下限: 0.3·m·g
```

8.2 推力下限クリップの効果

$$T_{cmd_min} = 0.3 \cdot m \cdot g \approx 0.59 \text{ N}$$

PID 切替時に $w = -2.4 \text{ m/s}$ (上昇中) が来ると $T_{fb} = K_d \cdot w = -6 \text{ N} \rightarrow$ 従来は $T_{cmd} \rightarrow 0$ で自由落下。下限クリップにより最低 0.59 N を維持し振動を防止。

8.3 t_cmd_to_u_hover

```
alive = eta > 1e-3
per = clip(T_cmd / (n_alive · T_max), 0, U_max)
u_hover_raw[k] = per if alive[k] else 0
```

故障でモータが減ると n_alive が減り per が自動的に増加 → 生存機で重力補償を分担。

8.4 U_ref への LPF (control フェーズのみ)

```
u_hover = alpha_lpf * u_hover_raw + (1.0 - alpha_lpf) * u_hover_prev
U_ref = np.tile(u_hover, params.N)
```

- **u_hover_raw**: PID が要求する即時ホバー duty
- **u_hover**: NMPC が参照する平滑化済み動作点 (ログの **T_cmd** は LPF 前の推力のまま)
- **$\tau = 0.10$ s**: 約 100 ms で 63% 追従 (外乱応答と故障過渡のバランス)

9. DisturbanceManager

```
schedule = [
    {
        't_start': float, 't_end': float,
        'kind': 'force_world' | 'torque_body',
        'axis': 0 | 1 | 2,
        'magnitude': float # [N] or [N·m]
    }, ...
]
```

区間は半开区間 [**t_start**, **t_end**)。複数エントリは加算。

10. AdmmNmpcHex (NMPC コア)

sample/nmpc.cpp の Python 鏡像。決定変数は全ホライズン入力 $U \in \mathbb{R}^{(N \cdot NU = 120)}$ 。

10.1 問題の定式化 (凝縮形)

線形化予測:

$$X \approx S_x \cdot x_0 + S_u \cdot U \quad (S_x: 100 \times 5, S_u: 100 \times 120)$$

コスト (対角 $\bar{Q}$, $\bar{R}$) :

$$J \approx \frac{1}{2} \|S_u \cdot U + S_x \cdot x_0 - X_{ref}\|^2_{\bar{Q}} + \frac{1}{2} \|U - U_{ref}\|^2_{\bar{R}}$$

10.2 step() の処理フロー

1. warm-start shift: U , z , λ を 1 段シフト (末尾ホールド)
2. S_x , S_u の更新:
 - `need_full_rebuild` or `cycle` $\geq$ `reset_period` $\rightarrow$ フル再構築 ($O(N \cdot NX^2 \cdot NU)$)

- それ以外 → 非線形ロールアウト + TR1 (Broyden rank-1) で差分更新
- 3. H の更新:
 - フル再構築: $H = Su^T \bar{Q} Su + \bar{R} + \rho I \rightarrow \text{Cholesky}$
 - 差分: SR2 (rank-2) で H 直接更新 → Cholesky
失敗時は自動でフル再構築
- 4. 勾配: $g = Su^T \bar{Q} (Sx \cdot x_0 - X_{\text{ref}}) - \bar{R} \cdot U_{\text{ref}}$
- 5. ADMM (1 iter): $U = H^{-1}(-g + \rho(z - \lambda))$, $z = \text{clip}(U + \lambda, \text{lo}, \text{hi})$, $\lambda += U - z$
- 6. 出力: $z[:\text{NU}]$ を box にクリップして返す

10.3 C++ との主な差分

項目	sample/nmpc.cpp	Python 実装
SR2 更新	LLT に rank±1 直適用	H に直接加算 → <code>np.linalg.cholesky</code>
線形ソルブ	Cholesky 因子バックサブ	<code>np.linalg.solve(H, rhs)</code>
数値型	float (想定)	float64

10.4 FDI 既知時の NMPC 更新 (今バージョン)

```
# η≈0 (完全故障) のみ u_max[k] = 0、部分故障は u_max[k] = 1.0 維持
if eta_t <= 1e-3:
    u_max[k] = 0.0
else:
    u_max[k] = U_max      # ∧ スケーリングのみで対応、u_max は縮減しない

nmpc_model.set_eta(fault_mgr.eta)    # B_eff 更新
nmpc.set_box(u_min, u_max)
nmpc.force_reset()                  # SR2 履歴リセット
```

11. FaultManager

11.1 スケジュール駆動 (旧来互換)

```
FaultManager(schedule=[(t_fail, motor_idx, eta_target), ...])
```

`t_fail` 到達で即時 (`hard_step=True`) または 1 次遅れで η を更新。

11.2 apply_faults (新機能)

```
events = fault_mgr.apply_faults([(4, 0.5)]) # 即時適用
```

`run_scenario` 内で絶対時刻 (`t_switch + delay`) に基づいて呼ばれる。戻り値の `events` を FDI ハンドラに渡してモデル更新を行う。

12. Logger と可視化

12.1 記録フィールド

フィールド	形状	内容
<code>t</code>	(T,)	時刻 [s] (1 kHz)
<code>x</code>	(T,7)	プラント状態
<code>u_cmd</code>	(T,6)	NMPC 出力または ascent PWM
<code>u_app</code>	(T,6)	実適用 (= $\eta \odot u_cmd$)
<code>eta</code>	(T,6)	モータ健全度
<code>iter_us</code>	(T,)	NMPC 計算時間 (非呼出ステップは前回値)
<code>was_reset</code>	(T,)	SR2 フル再構築フラグ
<code>z_ref</code>	(T,)	高度参照 (NED)
<code>T_cmd</code>	(T,)	PID 推力コマンド [N]
<code>T_trim</code>	(T,)	重力補償項 [N]
<code>T_fb</code>	(T,)	PID FB 項 [N]
<code>pid_iz</code>	(T,)	積分項 [N·s]
<code>F_d</code>	(T,3)	world 力外乱
<code>tau_d</code>	(T,3)	body トルク外乱

12.2 プロットレイアウト (5 行 × 4 列)

行	列0	列1	列2	列3
0	ϕ, ϕ_ref	θ, θ_ref	姿勢誤差	NMPC iter [μ s]
1	p [rad/s]	q [rad/s]	r [rad/s]	SR2 reset フラグ
2	u_cmd M1,M2	M3,M4	M5,M6	Σu_app
3	u_app M1-M3	M4-M6	$\eta_1 \sim \eta_6$	累積サチュ率
4	高度 (-z) vs z_ref_alt	w (NED)	$T_cmd / T_trim / T_fb$	$F_d_z + \tau_d$ (3 軸)

13. run_scenario (メインシミュレーションループ)

13.1 引数一覧

引数	型	説明
plan_open_loop_ascent	bool	True で 2 段階制御を有効化
plan_z_target	float None	目標高度 NED (デフォルト: params.z_ref_default)
plan_fault_delay_after_switch_s	float	PID 切替からの故障遅延 [s]
plan_faults	list	[(motor_idx, η), ...]
plan_phi_step	tuple None	(delay_s, phi_rad)
dist_mgr	DisturbanceManager None	外乱スケジュール
fdi_known	bool	True で故障時に NMPC モデル・box を更新

13.2.1 ステップの処理順 (各 dt_sim)

```
1. FaultManager.update(t)          →  $\eta$ , events (スケジュール駆動故障)
2. DisturbanceManager.get_wrench(t)→ F_world,  $\tau_{body}$ 
3. フェーズ遷移チェック:
   - ascent→control:  $z \leq z_{switch} \rightarrow alt\_pid.reset(), u_{hover\_prev} \leftarrow ascent$ 
   PWM, t_switch
   - control:  $t \geq t_{switch} + delay \rightarrow apply\_faults$  (**PID/LPF リセットなし**),
   t_fault
4. FDI ハンドラ (events & fdi_known) :
   → set_eta, set_box ( $\eta \approx 0$  のみ  $u_{max}=0$ ) , force_reset
5. 姿勢参照 x_ref 決定:
   - plan: 0 →  $t \geq t_{fault} + 5s$  で  $\phi_{ref}=0.2$ 
6. z_ref ランプ計算:
   - ascent: z_ref_t = z_target (ログ用)
   - control: 線形補間 z_at_switch → z_target (1.5s)
7. PID 更新 (control のみ) :
   → T_cmd → u_hover_raw → LPF → U_ref
   - ascent: U_ref=ascent PWM, u_hover_prev 追従
8. NMPC 呼出 (n%dec==0、control で u_cmd 採用) :
   → u_cmd 更新、ZOH
9. ascent: u_cmd = pwm_ascent (NMPC 出力無視)
10. u_applied =  $\eta \odot u_{cmd}$ 
11. PlantModel.step_rk4(dt_sim)
12. Logger.log(...)
```

13.3 compute_metrics の出力

メトリクス	意味
rms_phi, rms_theta	姿勢追従 RMS [rad]
rms_z_err	(z - z_ref) の RMS [m]

メトリクス	意味
peak_p/q/r	角速度ピーク [rad/s]
peak_w	w ピーク [m/s]
sat_rate	いずれかモータがサチュした割合
mean_iter_us, p50, p99	NMPC 計算時間 (NMPC 呼出ステップのみ)

14. sanity_checks (起動時 13 項目)

#	テスト	確認内容
1	B_c 表示	ロール/ピッチ/ヨー列の符号
2	M1 単独印加	$\dot{p} < 0$ (右モータ → 左ロール)
3	M5 単独印加	$\dot{q} > 0$ (前右モータ → 前上がりピッチ)
4	全機ホバー	角加速度 ≈ 0
5	総推力	$\Sigma(u_i \cdot T_{\max}) = m \cdot g$
6	B_c ランク	$\text{rank}(B_c[2:5,:]) = 3$
7	set_eta	$B_{\text{eff}}[:,k] / B_{\text{nom}}[:,k] = \eta_k$
8	PID ホバー	$T_{\text{cmd}} = m \cdot g$ ($z=z_{\text{ref}}, w=0$)
9	重力補償	$\phi=0.3$ で $T_{\text{trim}} = mg/\cos(\phi)$
10	アンチwindアップ	200 ms サチュで $i_z = 0$ 維持
11	PlantModel ホバー	$\dot{x} \approx 0$
12	DisturbanceManager	時間窓の開閉
13	t_cmd_to_u_hover	健全・1機故障時の per-motor 値

15. シナリオ一覧 (main)

共通タイムライン (全 8 シナリオ)

t = 0.000 s : 地上 z=0、ascent 開始 (全機 PWM=0.14)

t ≈ 1.443 s : z = -1.70 m (高度 1.70 m) で PID 切替 (決定論的)

t = 3.443 s : 故障注入 (切替+2.0 s)

t = 8.443 s : $\phi_{\text{ref}} = 0.2$ rad ステップ (故障+5.0 s)

t = 15.000 s : シミュレーション終了

各シナリオの故障条件

ID	故障	FDI	外乱	外乱時刻
S1	なし	—	なし	—
S2a	M5 $\eta=0.5$	既知	なし	—
S2b	M5 $\eta=0.5$	未知	なし	—
S3a	M5 $\eta=0$	既知	なし	—
S3b	M5 $\eta=0$	未知	なし	—
S4	M5+M3 $\eta=0$	既知	なし	—
S6	なし	—	F_world_z=+0.5 N（鉛直突風）	$t \in [6,7)$ s
S7	なし	—	$\tau_{\text{body_x}}=+0.02$ N·m（ロール外乱）	$t \in [6,7)$ s
S8	M5 $\eta=0$	既知	$\tau_{\text{body_x}}=+0.02$ N·m	$t \in [8,9)$ s

注: S5（高度ステップのみ）は 2 m ホバー固定プランと重なるため削除済。

16. 実行結果と比較

参照実行: [img/20260525_192917/](#)（U_ref LPF $\tau=0.10$ s、故障時 PID リセットなし）

16.1 現行版サマリ

シナリオ	rms_φ	rms_θ	rms_z	peak w	sat [%]	mean iter [μs]
S1	0.260	0.095	0.820	3.76	28	856
S2a	0.179	0.195	0.949	2.38	33	843
S2b	0.140	0.156	0.797	3.89	23	859
S3a	0.093	0.194	1.030	2.63	41	857
S3b	0.314	0.235	1.236	2.46	37	869
S4	0.202	0.441	2.120	4.64	62	902
S6	0.179	0.236	0.719	3.75	34	807
S7	0.084	0.073	0.646	3.12	24	834
S8	0.127	0.201	2.924	9.31	46	838

16.2 版間比較（rms_φ [rad]）

シナリオ	① LPF なし（ベース）	② LPF $\tau=0.15$ + 故障時 PID リセット	③ 現行 $\tau=0.10$ 、リセットなし
S1	0.391	0.089	0.260
S2a	0.157	0.068	0.179

シナリオ	① LPF なし（ベース）	② LPF $\tau=0.15$ + 故障時 PID リセット	③ 現行 $\tau=0.10$ 、リセットなし
S3a	0.123	0.244	0.093
S3b	0.128	0.358	0.314
S6	0.116	0.705	0.179
S7	0.070	0.331	0.084
S8	0.176	0.107	0.127

16.3 解釈

観点	内容
LPF の効果	故障後の U_{ref} 急変抑制に有効（S3a: ② 0.244 → ③ 0.093）
故障時 PID リセット	採用しない。定常ホバー中の i_z 消失で S3a/S6/S7 が悪化したため
$\tau = 0.10\text{ s}$	0.15 s より外乱・高度応答が速い（S6 rms_z: 5.72→0.72 m 級に改善）
S4	rms_z 30 m 級（①） → 2.1 m （③）。 θ 制御限界は残る（rms_ $\theta \approx 0.44$ ）
S8	rms_z・peak w が③のみ大きい（故障+外乱の時間重畳）。要個別チューニング
計算時間	mean iter $\approx 800\text{--}900\text{ }\mu\text{s}$ → 実効 1100–1200 Hz （目標 100 Hz に余裕）

S4 について: M3+M5 同時完全故障は正ピッチトルクが失われる**構造的制御不能**事例。高度は LPF+2 段階上昇で大幅改善するが、姿勢（特に θ ）・サチュ率は限界付近。

17. 依存関係と実行

```
cd program/NMPC_simulation
python nmpc_simulation.py
```

依存: NumPy, Matplotlib のみ（SciPy 不使用）
出力: `img/<YYYYMMDD_HHMMSS>/sim_s*.png`（8 枚）

matplotlib バックエンド: `MPLBACKEND` 未設定時は Qt5Agg → TkAgg → MacOSX → Agg の順に試行。

18. 設計上の制限（スコープ外）

項目	理由
水平位置 (x, y)・ ψ	並進状態をモデル化していない
センサノイズ・遅延	将来タスク
質量・慣性の不確かさ	パラメータ感度解析は別タスク

項目	理由
連続風場モデル	現状は step/pulse のみ
C++ 自動コード生成	sample/ への反映は移植時に別途対応

19. 改訂履歴

日付	主な変更内容
2026-05-25 (初版)	マルチレート + 高度 PID + 外乱注入 実装
2026-05-25 (v2)	Λ スケーリング修正 (u_max 二重縮減→撤廃)、故障時刻をイベント駆動に
2026-05-25 (v3)	Open-loop 離陸、PID 再チューニング、T_cmd 下限、R_u=10、絶対時刻イベント
2026-05-25 (v4, 最新)	U_ref LPF 追加 (u_ref_lpf_tau=0.10)、ascent 切替時のみ u_hover_prev 初期化、故障時 PID/LPF リセットは実装しない

本レポートはnmpc_simulation.py (1566 行) のソース構造に基づく技術ドキュメントです。Pico 2 移植時はParams と sample/nmpc_params.hpp を突き合わせてください。